



## Especificação da Linguagem D++\*

### 1. Introdução

Já pensaram em como seriam as linguagens de programação se certas escolhas fossem feitas de forma diferente? Se, por exemplo, ao especificar o que seria o primeiro comando condicional, ao invés de usar a palavra *if* alguém pensasse em outra palavra no momento e isso se tornasse o padrão?

Tendo esse pensamento em mente, vamos implementar uma linguagem cujas expressões e palavras reservadas são levemente diferentes das que costumamos usar na maioria das linguagens de programação. A linguagem **D++** é uma linguagem que assume traços de linguagens orientada a objetos e imperativas - mas não todos. Suas características são descritas neste documento.

**D++**, obviamente, é uma linguagem experimental, então esta especificação é passível de adaptações. Em caso de modificações na especificação, versões atualizadas serão postadas via SIGAA (no tópico de aula “Definições do Projeto”) e notificações serão enviadas aos alunos e alunas.

As Seções 2, 3, 4 e 5 deste documento apresentam a especificação da linguagem. A Seção 6 contém informações sobre a avaliação e entregas. Serão anexados ao SIGAA exemplos de código fonte em **D++** (extensão .dpp).

### 2. Características principais e léxico

Classes:

- Suporte a herança e polimorfismo (fraco)

- Contém atributos e métodos

- Atributos das classes são tipos primitivos ou outras classes.

- O tempo de vida de um atributo é o tempo de vida da instância da classe à qual ele pertence

- Escopo das variáveis, objetos e constantes de um método da classe é o bloco onde foram declarados

- Métodos podem ser funções ou procedimentos

Tipos primitivos:

- A linguagem aceita os tipos **answer** e **number**.
- **number**: números inteiros ou reais, expressos no sistema decimal. Sendo um valor real, a parte decimal do número é separada por um ponto.
- **answer**: booleanos que podem assumir dois valores: **yes** e **no**.

\***D++** seria uma versão orientada a objeto de uma linguagem imperativa fictícia chamada **D**.

## Blocos

- Blocos são delimitados pelas palavras **start** e **finish**.

## Comentários:

- A linguagem aceita comentários de linha, indicados pelo símbolo `--` no início da linha
- A linguagem aceita comentários de bloco (possivelmente de múltiplas linhas) delimitados por `{- e -}`.
- O funcionamento dos comentários de bloco em **D++** são similares aos da linguagem C.  
Exemplo: o que acontece se você compilar `/* */` em C? Estudem como um compilador C reconhece o fim de um comentário de bloco.

## Estruturas de controle (vide funcionamento na Seção Semântico):

- **in case that:** estrutura similar ao 'se/senão'
- **as long as:** estrutura similar ao 'enquanto'

## Métodos:

- Funções (**function**): possui parâmetros e retorno de valor.
- Procedimentos (**procedure**): possui parâmetros, não tem retorno de valor.

## Regras para identificadores:

- Pode-se utilizar: letras maiúsculas, letras minúsculas e *underline* ('\_').
- Identificadores de objetos, variáveis e constantes devem começar com letras minúsculas (**ID**).
- Identificadores de classe devem começar com letras maiúsculas (**ID\_CLASSE**).
- Não são permitidos espaços em branco e outros caracteres (ex.: @, \$, +, -, ^, ` , ~ , % etc.).
- Identificadores não podem ser iguais às palavras reservadas ou operadores da linguagem.

## Tipos compostos:

- A linguagem não possui.
- Classes podem ter tipagem recursiva (conter, como atributos, objetos de seu próprio tipo), o que permitira a criação de listas ou estruturas de dados mais elaboradas.

## Operadores:

- Possui operadores aritméticos, relacionais e booleanos (tipo answer).
- Comandos são terminados com '.' (ponto-final).

## 3. Sintático

A gramática da linguagem foi escrita seguindo as seguintes convenções:

- Variáveis da gramática são escritas em letras minúsculas sem aspas;
- Lexemas são escritos entre aspas simples;
- Símbolos escritos em letras maiúsculas representam o lexema de um token do tipo especificado;
- Os operadores seguem um padrão similar ao visto em aula.

```

programa : genealogia? (def_classe)+

genealogia: relação ('&' relação)* '.'

relação: 'family' tipo_classe 'derives' 'from' tipo_classe

def_classe : 'family' tipo_classe 'start' atributos metodos 'finish'

atributos : ( dec_obj | dec_var | dec_cons )*
métodos: ( dec_procedimento | dec_funcao)*

dec_obj : 'object' tipo_classe ID '.'
dec_var : 'alterable' tipo_primitivo ID '<<' exp '.'
dec_cons : 'unalterable' tipo_primitivo ID '<<' exp '.'

tipo : tipo_classe | tipo_primitivo
tipo_classe: ID_CLASSE
tipo_primitivo : 'answer' | 'number'

dec_procedimento: ('>>')? 'procedure' ID '[' parametros ']' bloco_cmd
                | 'procedure' ID '[' parametros ']' '.'

dec_funcao : 'function' tipo ID '[' parametros ']' bloco_exp
            | 'function' tipo ID '[' parametros ']' '.'

parametros : & | parametro ( '|' parametro )*
parametro : tipo ID

comando :      'in' 'case' 'that' '(' exp ')' comando
            |  'in' 'case' 'that' '(' exp ')' comando 'otherwise' comando
            |  'as' 'long' 'as' '(' exp ')' comando
            |  ID '<<' exp '.'
            |  chamada '.'
            |  bloco_cmd

bloco_cmd : 'start' ( dec_obj | dec_var | dec_cons )* ( comando )* 'finish'

exp :  VALOR_REAL | VALOR_INTEIRO | 'yes' | 'no'
    |  ID
    |  chamada
    |  (ID '.' '>')* atributo
    |  'if' '(' exp ')' exp 'else' exp //operador ternário
    |  '(' exp ')'
    |  '-' exp
    |  exp '+' exp
    |  exp '-' exp
    |  exp '*' exp
    |  exp '/' exp
    |  exp '=' exp
    |  exp '<' exp
    |  exp '>' exp
    |  '!' exp
    |  exp 'and' exp
    |  exp 'or' exp
    |  bloco_exp

bloco_exp : 'start' ( dec_obj | dec_var | dec_cons )* exp 'finish'
chamada : (ID '.' '>')* ID '[' lista_exp ']'
atributo : ID
lista_exp : & | exp ( '|' exp )*

```

#### 4. Semântico:

- O ponto de entrada do programa é o procedimento precedido pelos caracteres '>>'. No máximo um procedimento deve ter essa marcação em um programa. A execução fictícia de um programa em D++ é iniciada com a criação automática de uma instância da classe onde o procedimento marcado está declarado e a chamada a este procedimento. Um programa sem esta marcação gera um erro de compilação.
- Não há modificadores de acesso nas classes. Por padrão, todos os métodos são públicos e todos os atributos são privados. Na herança, uma classe filha herda todos os atributos e métodos de sua classe mãe. Em D++, uma classe só pode ter uma classe como mãe.
- Classes que não são declaradas como filhas de nenhuma outra são implicitamente filhas da classe Root. A classe Root não pode ser instanciada e não tem atributos nem métodos próprios.
- Caso o atributo, método ou função pertença ao objeto em questão ou às suas classes ancestrais, ele pode ser chamado diretamente. Chamadas a métodos ou funções de outros objetos devem ser feitas usando o identificador deste outro objeto (exemplo: **obj->chamada[]**).
- Uma classe pode conter métodos abstratos. Neste caso, a classe deve conter apenas a assinatura do método, sem seu corpo. Elas adquirem um comportamento similar ao que acontece com classes abstratas, ou seja, não podem ser instanciadas. Uma classe filha só é considerada concreta se implementa todos os métodos abstratos de suas ancestrais.
- Não há sobrecarga de operadores nem de funções.
- Não há variáveis compostas.
- A prioridade dos operadores é igual à de C.
- Variáveis e atributos só podem ser usados se forem inicializados.
- Parâmetros sempre são passados por cópia (valor) em funções e procedimentos.
- Existe uma classe Peripherals com dois métodos pré-definidos:
  - **show[]** procedimento que recebe como argumento uma expressão e a imprime em tela.
  - **capture[]**: função que retorna o valor inserido pelo usuário no teclado.

Exemplo:

Nas declarações:

```
object Peripherals io.
```

No uso:

```
cont << io->capture[].
```

Obs: Assumam que os dois métodos são compatíveis com os tipos primitivos.

O que deve ser verificado na análise semântica:

- Se as entidades criadas pelo usuário são inseridas na tabela de símbolos - com os atributos necessários - quando são declaradas;
- Se uma entidade foi declarada e está em um escopo válido no momento em que ela é utilizada;
- Se entidades foram declaradas e definidas (inicializadas) quando isso se fizer necessário;
- Checar a compatibilidade dos tipos de dados envolvidos nas declarações, vínculos, chamadas de função e expressões. Inclui suporte a polimorfismo e herança nas classes criadas.

## 5. Geração de Código

- O código alvo do compilador pode ser Java ou C++.
- As entradas/saídas dos programas executáveis são definidas pela função que contém o marcador de ponto de entrada, e podem ser obtidas/exibidas nesta etapa como comandos de leitura e escrita em console.

## 6. Desenvolvimento do Trabalho

Trabalhos devem ser desenvolvidos em quartetos (preferencialmente), trios, duplas ou individualmente. Os grupos devem se cadastrar na planilha correspondente à turma dos seus componentes:

[https://docs.google.com/spreadsheets/d/1o3xKcgUFDcLqMFJu0W9PHNT8Inn33jZIEIVZwA\\_FOmE/edit](https://docs.google.com/spreadsheets/d/1o3xKcgUFDcLqMFJu0W9PHNT8Inn33jZIEIVZwA_FOmE/edit)

Prazo para preenchimento da planilha: 16/04/2026

Serão abertos fóruns no SIGAA para a discussão sobre as etapas. Em caso de dúvida, verifique inicialmente no fórum se ela já foi resolvida. Se ela persiste, consulte os monitores ou a professora.

### 6.1. Ferramentas

- Implementação com SableCC, linguagem Java.
- IDE Java (recomendação: Eclipse).
- Submissão das etapas do projeto via SIGAA. Haverá uma tarefa para cada etapa.

### 6.2. Avaliação

- A avaliação será feita com base nas etapas entregues e em entrevistas feitas com os grupos durante as aulas de acompanhamento do projeto.
- O valor de cada etapa está definido no plano de curso da disciplina.
- O cumprimento das requisições de formato também será avaliado na nota de cada etapa.

### 6.3. Etapas

#### Etapa 1. Códigos D++

- **Prazo:** 22/04/2026
- **Atividade:** escrever três códigos em D++ que, unidos, usem todas as alternativas gramaticais (ou seja, todos os recursos) da linguagem.
- **Formato de entrega:** arquivo comprimido contendo três códigos, onde cada código deve estar escrito em um arquivo de texto simples, com extensão “.dpp”.

#### Etapa 2. Análise Léxica

- **Prazo:** 11/05/2026
- **Atividade:** implementar analisador léxico da linguagem com o SableCC, fazendo a impressão dos lexemas e tokens reconhecidos ou imprimindo erro quando o token não for reconhecido.
- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser grupo\_X.sable, onde X é o número do grupo (vide planilha de cadastro de grupos). O nome do pacote a ser gerado pelo sablecc deve se chamar **dplusplus** (em letras minúsculas).

#### Etapa 3. Análise Sintática

- **Prazo:** 08/06/2026
- **Atividade:** implementar analisador sintático da linguagem com o SableCC, fazendo impressão da árvore sintática em caso de sucesso ou impressão dos erros caso contrário.
- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser grupo\_X.sable, onde X é o número do grupo (vide planilha de cadastro de grupos).

O nome do pacote a ser gerado pelo sablecc deve se chamar **dplusplus** (em letras minúsculas).

#### **Etapa 4. Análise Sintática Abstrata**

- **Prazo: 01/07/2026**
- **Atividade:** implementar analisador sintático abstrato em SableCC, com impressão da árvore sintática.
- **Formato de entrega:** apenas o arquivo .sable deve ser enviado. O nome do arquivo deve ser grupo\_X.sable, onde X é o número do grupo (vide planilha de cadastro de grupos). O nome do pacote a ser gerado pelo sablecc deve se chamar **dplusplus** (em letras minúsculas).

#### **Etapa 5. Tabela de Símbolos e Análise Semântica**

- **Prazo: 15/07/2026**
- **Atividade:** implementação da tabela de símbolos, validar escopo, declaração e definição de identificadores. Implementar verificação de tipos (incluindo polimorfismo e herança).
- **Formato de entrega:** projeto completo, incluindo obrigatoriamente: o arquivo .sable; todas as classes java escritas pelo grupo ou geradas automaticamente; e arquivos .dpp que demonstrem o que foi feito nesta tarefa.  
Também é obrigatória a entrega de um pdf contendo uma breve explicação sobre o que foi implementado nesta etapa e como.

#### **Etapa 6. Geração de código (extra: 2 pontos na segunda unidade):**

- **Prazo: o mesmo da Etapa 5**
- Compilação de código **D++** com geração de código em linguagem alvo.

#### **Observações importantes:**

- Entregas após o prazo sofrem penalidade de metade da nota da etapa por dia de atraso.
- Trabalhos entregues com atraso devem ser submetidos na Tarefa 'Entrega após prazo', no SIGAA, que ficará aberta durante todo o período.
- Arquivos enviados por e-mail não serão considerados.

**Bom trabalho!**